

Laboratorio Semana 8 – Cifrado AES-256, HMAC-SHA256, SSH y Git

V

Nombre: Luis de Leon

Objetivo Aplicar cifrado simétrico para proteger la confidencialidad de un archivo, utilizar HMAC-SHA256 como huella autenticada y trabajar con un repositorio compartido de GitLab mediante SSH.

Herramientas: PowerShell + Git + OpenSSL

Regla de seguridad: Nunca subas la contraseña o clave secreta al repositorio de GitLab.

1. Conexión al repositorio mediante SSH Verifica

Git: `git --version`

Prueba la conexión: `ssh -T git@github.com`

Si es la primera conexión, verifica el fingerprint del servidor antes de aceptarlo. Clona el repositorio compartido: `git clone git@github.com:[tu_repo]_git`

Entra al repositorio y crea tu rama: `cd git checkout -b lab-[nombre_branch]`

2. Crear un archivo personalizado

Crea un archivo cuyo nombre incluya tu nombre y apellido. Cada estudiante debe utilizar contenido personalizado.

Ejemplo: `mensaje_LuisRojas.json` Contenido:

```
{  
  "nombre": "TU NOMBRE",  
  "curso": "Manejo e Implementación de Archivos",  
  "carne": "[tu_carne]",  
  "archivo": "mensaje_[Nombre_apellido].txt"  
}
```

3. Cifrar el archivo con AES-256

Utiliza **OpenSSL** para cifrar tu archivo con **AES-256-CBC**. El archivo resultante tendrá extensión `.enc`

3.1 Verifica OpenSSL

Ejecuta el siguiente comando en **PowerShell**: `openssl version`

Si OpenSSL no está instalado, ejecuta: `winget install --id ShiningLight.OpenSSL.Light --exact` Si instalaste OpenSSL, **reinicia VS Code** antes de continuar.

3.2 Comprueba instalación

`Get-ChildItem "C:\Program Files\OpenSSL-Win64\bin\openssl.exe"`

3.3 Cifra tu archivo

Comprueba dónde quedó instalado: `Get-ChildItem "C:\Program Files\OpenSSL-Win64\bin\openssl.exe"`

Agrega OpenSSL al PATH de tu usuario: `$opensslPath = "C:\Program Files\OpenSSL-Win64\bin"`

`[Environment]::SetEnvironmentVariable("Path", $env:Path + ";" + $opensslPath, "User")`

Actualiza el PATH de esta terminal inmediatamente: `$env:Path += ";C:\Program Files\OpenSSLWin64\bin"`

3.4 Cifra tu archivo

Ejecuta: `openssl enc -aes-256-cbc -salt -pbkdf2 -iter 100000 -in .\mensaje_TuNombre.json -out .\mensaje_TuNombre.enc`

V

Cuando OpenSSL solicite la contraseña, introduce una contraseña que solo tú conozcas. Deberás introducirla dos veces. **NO escribas la contraseña en este laboratorio ni la subas a GitHub.**

3.5 Comprueba que se creó el archivo: Ejecuta: `Get-ChildItem`

Deben aparecer, como mínimo: `mensaje_TuNombre.txt` `mensaje_TuNombre.enc`

3.6 Comprueba que el archivo cifrado no muestra directamente el mensaje:

`Get-Content .\mensaje_TuNombre.enc` ¿Qué observaste?

4. Descifrar y recuperar el archivo

Utiliza la misma contraseña para recuperar el contenido original.

`openssl enc -d -aes-256-cbc -pbkdf2 -iter 100000 -in .\mensaje_TuNombre.enc -out .\mensaje_TuNombre_recuperado.json`

Comprueba el contenido recuperado: `Get-Content .\mensaje_TuNombre_recuperado.json`

5. Generar una huella autenticada con HMAC-SHA256

Ahora generarás una huella que depende del archivo y de una clave secreta. Esto permite comprobar integridad y autenticidad cuando quien verifica también conoce la clave.

archivo + clave secreta → HMAC-SHA256 → huella

corre: `openssl dgst -sha256 -hmac "TU_CLAVE_SECRETA" .\mensaje_LuisRojas.json` ¿Cuál es el HMAC-SHA256 obtenido?

6. Hash vs. cifrado vs. HMAC

Mecanismo	Objetivo	Clave	Ejemplo
SHA-256	Integridad	No	Huella
AES-256 (encrypt)	Confidencialidad	Sí	Archivo .enc
HMAC-SHA256	Integridad + autenticidad	Sí	Huella autenticada
SSH	Comunicación/autenticación segura	Sí	GitLab

Pregunta 1: Si alguien obtiene tu archivo .enc pero no conoce la contraseña, ¿qué propiedad estás protegiendo?
La confidencialidad de la información.

Pregunta 2: ¿Por qué no debes subir la contraseña a GitLab?

Porque cualquier persona que obtenga la contraseña podría descifrar el archivo y acceder a su contenido.

Pregunta 3: ¿Qué diferencia principal existe entre SHA-256 y HMAC-SHA256?

SHA-256 genera una huella sin utilizar una clave secreta, mientras que HMAC-SHA256 utiliza una clave secreta y permite verificar integridad y autenticidad.

7. Entrega

Dentro del repositorio de GitHub, crea la carpeta semana-8/lab8/ y coloca en ella un PDF con la solución del laboratorio, incluyendo las respuestas, el HMAC-SHA256 obtenido y pantallazos como evidencia de los comandos y procedimientos realizados. También incluir el archivo .json creado y el archivo .enc generado durante el cifrado. Enviar en la plataforma el enlace de la carpeta lab8 como evidencia de la entrega.

```
PS C:\> ssh-keygen -t ed25519 -C "lpdeleonh@correo.url.edu.gt"
Generating public/private ed25519 key pair.
Enter file in which to save the key (C:\Users\luisd\.ssh/id_ed25519):
Created directory 'C:\\Users\\luisd\\.ssh'.
Enter passphrase (empty for no passphrase):
Enter same passphrase again:
Your identification has been saved in C:\Users\luisd\.ssh/id_ed25519
Your public key has been saved in C:\Users\luisd\.ssh/id_ed25519.pub
The key fingerprint is:
SHA256:iNeqifUlqot6mDbVUyWUabhbfofSLIzyJlIsAhaPnPS8 lpdeleonh@correo.url.edu.gt
The key's randomart image is:
PS C:\url-mia-2026> cd url-mia-2026
PS C:\url-mia-2026\url-mia-2026> git git checkout -b lab-8-Luis-deLeon
git: 'git' is not a git command. See 'git --help'.

The most similar command is
    init
PS C:\url-mia-2026\url-mia-2026> git checkout -b lab-8-Luis-deLeon
Switched to a new branch 'lab-8-Luis-deLeon'
PS C:\url-mia-2026\url-mia-2026> notepad .\mensaje_LuisDeLeon.json
PS C:\url-mia-2026\url-mia-2026> Get-ChildItem

PS C:\Users\luisd> cd ..
PS C:\Users> cd ..
PS C:\> cd ..
PS C:\> git --version
git version 2.55.0.windows.2
PS C:\> ssh -T git@github.com
Hi lpdh86! You've successfully authenticated, but GitHub does not provide shell access.
PS C:\> git clone git@github.com:lpdh86/url-mia-2026.git
fatal: destination path 'url-mia-2026' already exists and is not an empty directory.
PS C:\> cd url-mia-2026
PS C:\url-mia-2026> git clone git@github.com:lpdh86/url-mia-2026.git
Cloning into 'url-mia-2026'...
remote: Enumerating objects: 59, done.
remote: Counting objects: 100% (59/59), done.
remote: Compressing objects: 100% (54/54), done.
remote: Total 59 (delta 6), reused 3 (delta 0), pack-reused 0 (from 0)
Receiving objects: 100% (59/59), 799.92 KiB | 2.64 MiB/s, done.
Resolving deltas: 100% (6/6), done.
PS C:\url-mia-2026> dir
```

||||

Directorio: C:\url-mia-2026\url-mia-2026

Mode	LastWriteTime	Length	Name
d-----	28/08/2026 8:40		Laboratorio_2_CSV_JSON
d-----	28/08/2026 8:40		semana8
-a----	28/08/2026 8:40	44147	Lab 2 MIA.pdf
-a----	28/08/2026 9:17	32	mensaje_LuisDeLeon.enc
-a----	28/08/2026 8:53	0	mensaje_LuisDeLeon.json
-a----	28/08/2026 9:23	0	mensaje_LuisDeLeon_recuperado.json
-a----	28/08/2026 8:40	56386	MIA lab 7.pdf
-a----	28/08/2026 8:40	460918	MIA_LaboratorioNo.3_Semana_3.pdf
-a----	28/08/2026 8:40	126	README.md
-a----	28/08/2026 8:40	899	script_lab3.ps1

```
PS C:\url-mia-2026\url-mia-2026> openssl dgst -sha256 -hmac "Pablo2018.$" .\mensaje_LuisDeLeon.json
HMAC-SHA2-256(.\mensaje_LuisDeLeon.json)= ad697ccc36ee6e85463de22a5250d23cc000e0e4e440400f6c8bc1b9dfc9fba6
PS C:\url-mia-2026\url-mia-2026> |
```

```
PS C:\> Get-ChildItem "C:\Program Files\OpenSSL-Win64\bin\openssl.exe"
```

Directorio: C:\Program Files\OpenSSL-Win64\bin

Mode	LastWriteTime	Length	Name
-a----	25/08/2026 9:21	821760	openssl.exe

```
PS C:\> cd url-mia-2026
PS C:\url-mia-2026> cd url-mia-2026
PS C:\url-mia-2026\url-mia-2026> $opensslPath = "C:\Program Files\OpenSSL-Win64\bin"
PS C:\url-mia-2026\url-mia-2026> [Environment]::SetEnvironmentVariable("Path", $env:Path + ";" + $opensslPath, "User")
PS C:\url-mia-2026\url-mia-2026> $env:Path += ";C:\Program Files\OpenSSL-Win64\bin"

PS C:\url-mia-2026\url-mia-2026> openssl enc -aes-256-cbc -salt -pbkdf2 -iter 100000 -in .\mensaje_LuisDeLeon.json -out
.\mensaje_LuisDeLeon.enc
enter AES-256-CBC encryption password:

Verifying - enter AES-256-CBC encryption password:

PS C:\url-mia-2026\url-mia-2026> dir
```

Directorio: C:\url-mia-2026\url-mia-2026